S2D-04: Alert Migration - Anomaly Detectors

> **Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 4 of 9 | **Created:** January 2026 | **Last Updated:** 10/05/2026

Overview

This notebook explains how to translate Splunk alert conditions into Dynatrace Anomaly Detectors. Understanding the fundamental differences between scheduled queries and continuous monitoring is essential for successful alert migration.

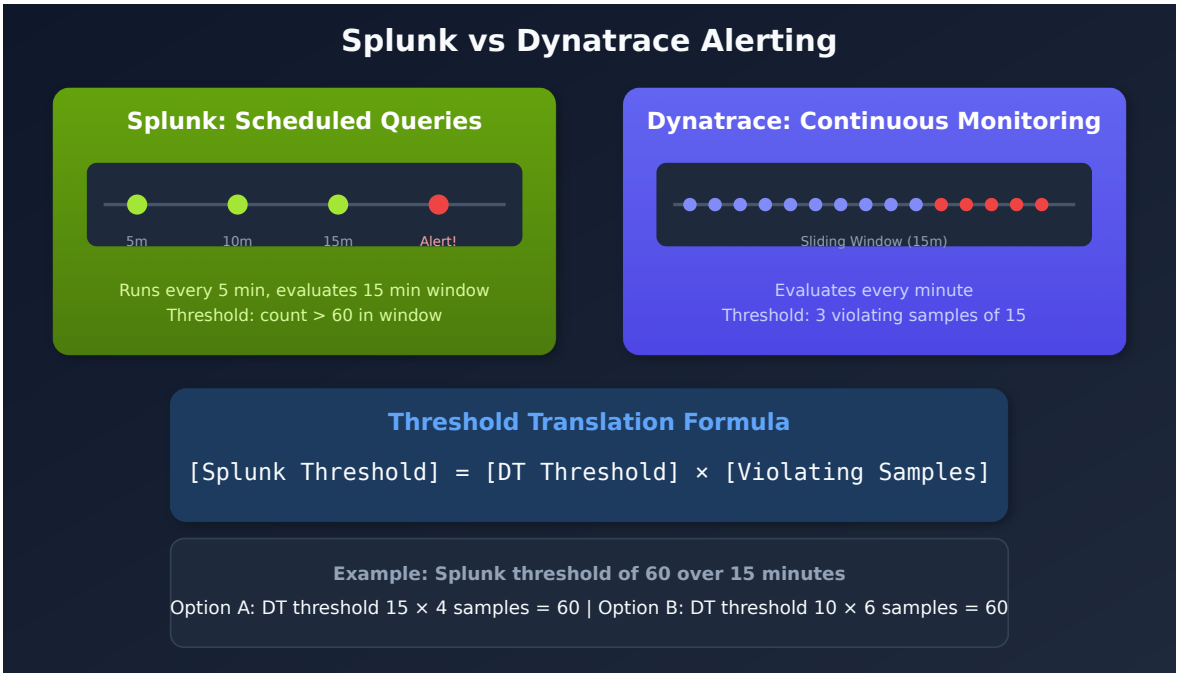


Table of Contents

1. Splunk vs Dynatrace Alerting
2. The Sliding Window Concept

3. Threshold Translation Formula
4. Migration Strategies
5. Concept Mapping
6. Creating an Anomaly Detector
7. Migration Decision Tree
8. Validation Query

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`logs.read`, `settings.write`
Knowledge	Understanding of source Splunk alert logic

Learning Objectives

By the end of this notebook, you will be able to:

1. Explain the difference between Splunk scheduled alerts and Anomaly Detectors
2. Apply the threshold translation formula
3. Choose the appropriate migration strategy for different alert types
4. Configure Anomaly Detector parameters correctly

Splunk vs Dynatrace Alerting

Splunk Alerting Model

In Splunk, alerts are **scheduled queries** that:
- Run at defined intervals (e.g., every 5 minutes)
- Evaluate a query over a timeframe (e.g., last 15 minutes)

- Trigger when results exceed a threshold (e.g., count > 10)
- Apply suppression to prevent repeated alerts

Dynatrace Alerting Model

Anomaly Detectors provide **continuous monitoring** that:
- Evaluate every minute (1-minute granularity)
- Use a sliding window approach
- Require multiple violating samples to trigger
- Require dealerting samples to close

Key Parameters

Parameter	Description	Maximum
Threshold	Value triggering a violation	Unlimited
Sliding Window	Time range evaluated	60 minutes
Violating Samples	Minutes above threshold to alert	60
Dealerting Samples	Minutes below threshold to close	60

The Sliding Window Concept

Anomaly Detectors continuously evaluate metrics using a sliding window.

Example Configuration:
- Metric: Error Log Count
- Threshold: 5 errors/minute
- Sliding Window: 15 minutes
- Violating Samples: 3
- Dealerting Samples: 5

Behavior:
1. Every minute, the detector checks if error count > 5
2. If 3 of the last 15 minutes exceed the threshold → Alert opens
3. Alert closes once at least 5 samples in the window are back below threshold *and* the number of violating samples has fallen below 3

Threshold Translation Formula

The key insight for translation is that both platforms should require roughly the same number of events to trigger:

```
[Splunk Threshold] = [Dynatrace Threshold] × [Dynatrace Violating Samples]
```

Example Translation

Splunk Alert:
- Runs every 5 minutes
- Timeframe: last 15 minutes
- Threshold: 60 error logs
- Suppress: 15 minutes

Dynatrace Options (all satisfy 60 total errors):

Option	DT Threshold	Violating Samples	Product
A	15	4	60
B	10	6	60
C	30	2	60
D	20	3	60

Migration Strategies

Option 1: Alert Reimagined (Recommended)

Strategy: Calculate new threshold and violating samples to most closely replicate Splunk behavior.

Parameters:
- Threshold: 3
- Sliding Window: 15
- Violating Samples: 2 (triggers at 6 total errors spread across 2 minutes)
- Dealerting Samples: 15

Pros:
- Best balance between sensitivity and stability
- Closest to expected alert behavior

Cons:
- Requires threshold recalculation
- Mindset shift for teams used to Splunk

Custom-alert query (paste this version into the alert; it has no `from:` / `to:`, see Step 1 below):

```
// Paste into the custom alert WITHOUT from:/to: — the alert supplies its own
sliding window
fetch logs
| filter status == "ERROR"
| filter matchesPhrase(k8s.cluster.name, "production")
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries count = count(), by:{dt.entity.cloud_application},
interval:1m
```

The cell below is the same query with `from:-24h`, so you can preview it in a notebook.

```
// Option 1: Alert Reimagined - notebook PREVIEW of the custom-alert query
above
// Remove from:-24h when you paste it into the custom alert
// Threshold: 3, Violating Samples: 2, Sliding Window: 15
fetch logs, from:-24h
| filter status == "ERROR"
| filter matchesPhrase(k8s.cluster.name, "production")
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries count = count(), by:{dt.entity.cloud_application},
interval:1m
```

Option 2: Conservative Threshold

Strategy: Use a 1-minute threshold equal to Splunk threshold, with high violating samples.

Parameters:
- Threshold: 6 (same as Splunk total)
- Sliding Window: 15
- Violating Samples: 1
- Dealerting Samples: 15

Pros:
- Easy to understand and explain
- Preserves original threshold number

Cons:
- Very sensitive to brief spikes
- May alert more frequently than Splunk

The custom-alert query is the same as Option 1 (no `from:` / `to:`); only the detector settings change. The cell below is the notebook preview.

```
// Option 2: Conservative - notebook PREVIEW (remove from:-24h in the custom
alert)
// Threshold: 6, Violating Samples: 1, Sliding Window: 15
fetch logs, from:-24h
| filter status == "ERROR"
| filter matchesPhrase(k8s.cluster.name, "production")
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries count = count(), by:{dt.entity.cloud_application},
interval:1m
```

Option 3: Aggressive Threshold

Strategy: Divide Splunk threshold by sliding window for per-minute threshold.

Parameters:
- Threshold: 0.4 (6 ÷ 15 = 0.4 per minute)
- Sliding Window: 15
- Violating Samples: 15 (all samples must violate)
- Dealerting Samples: 15

Pros:
- Very stable, requires sustained issues
- Less sensitive to spikes

Cons:
- May miss short-duration issues
- Delayed alerting compared to Splunk

Option 4: Data-Driven

Strategy: Analyze actual data patterns to determine optimal thresholds.

Steps:
1. Query historical data at the grain the detector evaluates (1-minute samples) to understand baseline patterns
2. Identify normal vs. problematic periods
3. Set thresholds that would have caught past incidents
4. Tune based on observed alert frequency

Pros:
- Most accurate for the specific environment
- Data-backed decisions

Cons:
- Requires more analysis effort
- Needs historical data availability

```
// Data-driven threshold: per-minute error distribution (detectors evaluate 1-minute samples)
// Keep the window at 24h or less: over longer windows interval:1m is coarsened silently
fetch logs, from:now()-24h
| filter status == "ERROR"
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries count = count(), interval:1m
| fieldsAdd per_min_avg = arrayAvg(count), per_min_p95 = arrayPercentile(count, 95), per_min_max = arrayMax(count)
```

Concept Mapping

Splunk Concept	Dynatrace Equivalent	Notes
Query Timeframe	Sliding Window	Both define evaluation period
Suppress / throttle	No direct equivalent	An open event absorbs repeat violations until it closes (de-alerting samples); re-notification is controlled in the routing workflow
Threshold	Threshold × Violating Samples	Must calculate product

Splunk Concept	Dynatrace Equivalent	Notes
Schedule Frequency	Delay (default 1 minute, up to 60)	Evaluation stays per-minute; Delay only batches executions

The configuration page says: *"While the default delay period between custom alert executions is 1 Minute, you can configure the query execution to longer intervals"*, and *"the delay can't be longer than 60 Minutes."* On de-alerting: *"The number of violating samples must be lower than the threshold number to raise the problem. The number of "normal" samples must be greater than or equal to the number of de-alerting samples. Both criteria must be met to close the event."*

Creating an Anomaly Detector

Step 1: Create the DQL Query

Your query must return timeseries data with a 1-minute interval.

Do not include `from:` or `to:` in the query you paste into the custom alert. The DQL writing guide says: *"Don't use from: and to: when creating a DQL query for the custom alert. Otherwise, you'll get failures whenever the custom alert tries to execute your query."* The alert applies its own sliding window. Keep `from:` only in the notebook preview.

Custom-alert template:

```
// Paste into the custom alert WITHOUT from:/to:
fetch logs
| filter status == "ERROR"
| filter matchesPhrase(k8s.cluster.name, "your-cluster")
| filter matchesPhrase(k8s.namespace.name, "your-namespace")
| makeTimeseries count = count(), by:{dt.entity.cloud_application},
interval:1m
```

Notebook preview of the same template:

```
// Notebook PREVIEW of the custom-alert template above
// Remove from:-24h when you paste it into the custom alert
fetch logs, from:-24h
| filter status == "ERROR"
| filter matchesPhrase(k8s.cluster.name, "your-cluster")
| filter matchesPhrase(k8s.namespace.name, "your-namespace")
| makeTimeseries count = count(), by:{dt.entity.cloud_application},
interval:1m
```

Step 2: Configure Detector Settings

Setting	Value	Rationale
Name	`[AppName] High Error Count`	Clear identification
Threshold	Calculated per formula	See translation formula
Sliding Window	Match Splunk timeframe	Usually 15-60 minutes
Violating Samples	Depends on strategy	See migration options
Dealerting Samples	How many back-to-normal samples close the event	Prevents alert flapping (not the same as Splunk suppress)

Step 3: Configure Event Properties

Property	Example	Purpose
Event Name	`[AppName] P2 - {dims:host.name} - High Errors`	Alert identification
Event Type	`ERROR_EVENT`	Severity classification
Description	Dynamic with affected entity	Context for responders

Placeholders take the form `{dims:<dimension>}`. The configuration page says: *"{dims} - a list of all dimensions (and their values) of the metric that violated the threshold. You can also specify a particular dimension: {dims:dt.entity.}."* Type `{` in the Event name field to list the placeholders available for your query (*"To see available placeholders, type { in the input field."*).

Migration Decision Tree

Use this decision tree to choose your migration approach:

1. **Is the Splunk alert timeframe > 1 hour?**
   - Yes → Use Workflows (see S2D-05) or ArrayMovingSum (see S2D-06)
   - No → Continue
2. **Is the alert scheduled infrequently (< 6 times/day)?**
   - Yes → Consider Workflows (see S2D-05)
   - No → Continue
3. **Do you have historical data for analysis?**
   - Yes → Use Option 4 (Data-Driven)
   - No → Continue
4. **Is minimizing false positives critical?**
   - Yes → Use Option 1 (Alert Reimagined) or Option 3 (Aggressive)
   - No → Use Option 2 (Conservative)

Validation Query

Before creating an Anomaly Detector, validate your query returns expected results:

```
// Validate alert query returns timeseries data (notebook preview)
// Remove from:now()-24h and the limit when you paste it into the custom alert
fetch logs, from:now()-24h
| filter status == "ERROR"
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries count = count(), by:{dt.entity.cloud_application},
interval:1m
| limit 100
```

Next Steps

- **S2D-05: Alert Migration - Workflows** - For alerts that don't fit the Dynatrace Intelligence model
- **S2D-06: ArrayMovingSum** - For timeframes exceeding 1 hour

References

- anomaly-detection (DT docs)
- Anomaly detection metric events (DT docs)
- Anomaly detection configuration (DT docs)
- Anomaly Detection DQL writing guide (DT docs)
- Alerting profiles — classic surface (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.